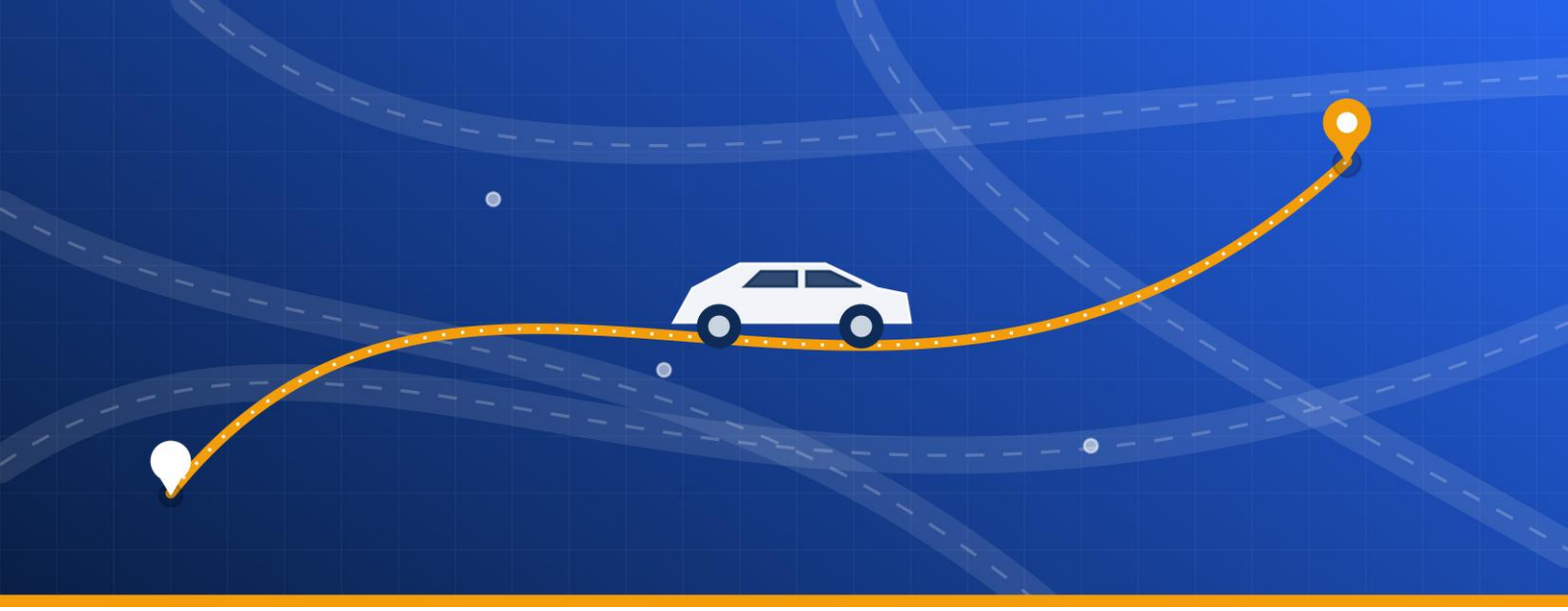


SERVICIO NACIONAL DE APRENDIZAJE · SENA

Centro de la Industria, la Empresa y los Servicios (CIES) — Regional Huila

Drivique Location

Documento de Especificación de Arquitectura de Software, Diseño y Selección de Frameworks

Plataforma Integral de Gestión y Alquiler de Vehículos

TECNÓLOGO EN ANÁLISIS Y DESARROLLO DE SOFTWARE (ADSO) · FICHA 3145555

PROYECTO FORMATIVO	**DRIVIQUE LOCATION (Gestión de Flota, Reservas y Contratos de Alquiler)**
INTEGRANTES / EQUIPO DE DESARROLLO	Laura Vanessa Pérez Perdomo Danna Valentina Barrios Penagos Emily Sharith Amézquita Saavedra
INSTRUCTORES EVALUADORES	Instructor Carlos Julio Cadena Instructor Pinto
PROGRAMA DE FORMACIÓN	Tecnólogo en Análisis y Desarrollo de Software (ADSO)
FICHA DE CARACTERIZACIÓN	3145555
INSTITUCIÓN EDUCATIVA	Servicio Nacional de Aprendizaje (SENA)
CENTRO DE FORMACIÓN	Centro de la Industria, la Empresa y los Servicios (CIES)
CIUDAD Y AÑO	Neiva, Huila — 2026
VERSIÓN DEL DOCUMENTO	**2.0 — Especificación Definitiva de Arquitectura, Frameworks y Patrones**

Tabla de Contenido

Organización capitular de la documentación arquitectónica y técnica del sistema Drivique.

1. INTRODUCCIÓN Y MARCO GENERAL 3
1.1 Propósito Formal del Documento 3
1.2 Alcance del Sistema y Distribución por Plataformas 3
1.3 Relación con el Proyecto Formativo SENA 4
2. ANÁLISIS INTEGRAL DE INSUMOS DEL PROYECTO 5
2.1 Análisis de la Especificación de Requisitos de Software (SRS) 5
2.2 Análisis del Modelado de Dominio y Base de Datos 6
2.3 Análisis de Mockups e Interfaces de Figma (+32 Pantallas) 6
2.4 Análisis de Interacciones, Eventos y Flujos de Usuario 6
3. SELECCIÓN Y JUSTIFICACIÓN DEL STACK TECNOLÓGICO 8
3.1 Frontend Web: Evaluación Multicriterio (React 19 + Vite 8) 8
3.2 Frontend Móvil: Evaluación Multicriterio (React Native 0.86 + Expo 57) 8
3.3 Backend API REST: Evaluación Multicriterio (Java 17 + Spring Boot 4) 9
3.4 Persistencia y Migraciones: PostgreSQL 17 + Liquibase 10
3.5 Ficha Técnica Consolidada de Dependencias y Versiones 10
4. MODELO Y ESTILO ARQUITECTÓNICO DEL SISTEMA 12
4.1 Modelo C4 - Nivel 1: Contexto del Sistema Drivique 12
4.2 Modelo C4 - Nivel 2: Contenedores y Límites Arquitectónicos 12
4.3 Flujo de Capas y Arquitectura Interna de los Frontends 13
5. PATRONES DE DISEÑO DE SOFTWARE IMPLEMENTADOS 15
5.1 Patrones Estructurales 15
5.2 Patrones de Comportamiento 17
5.3 Patrones Creacionales y de Arquitectura Backend 19
5.4 Antipatrones Identificados y Estrategias de Prevención 21
6. ESTÁNDARES DE DISEÑO Y CONTRATOS DE LA API REST 22
7. MODELO DE DATOS Y ARQUITECTURA DE PERSISTENCIA 23
7.1 Diccionario de Esquemas y Entidades de Dominio 23
7.2 Estándares Transversales de Base de Datos 24
8. INTERFACES GRÁFICAS (UI/UX), ACCESIBILIDAD E INTERNACIONALIZACIÓN 25
8.1 Principios de Usabilidad de Nielsen Norman Group 25
8.2 Accesibilidad Web y Móvil (WCAG 2.1 AA) 25
8.3 Modo Oscuro y Sistema de Tokens CSS 25
8.4 Soporte de Internacionalización i18n (5 Idiomas) 25
9. ESTRUCTURA DETALLADA DE CARPETAS DE LOS REPOSITORIOS 26
9.1 Frontend Web (front-end_web/web-drivique) 26
9.2 Frontend Móvil (front-end_movil/app-drivique) 27
9.3 Backend API REST (back-end) 27
10. CONCLUSIONES Y RECOMENDACIONES TÉCNICAS 28
10.1 Conclusiones de Ingeniería de Software 28
10.2 Recomendaciones Técnicas para el Equipo 28
11. ANEXOS Y MATRIZ DE TRAZABILIDAD (SRS vs. CÓDIGO) 29

1. INTRODUCCIÓN Y MARCO GENERAL

1.1 Propósito Formal del Documento

El presente documento técnico constituye la especificación formal y definitiva del diseño arquitectónico, la selección multicriterio de frameworks y librerías, los patrones de diseño de software implementados, el modelo de datos relacional y los estándares de interfaz de usuario para el sistema de información Drivique Location.

Su objetivo primordial es erigirse como la "fuente única de la verdad técnica" (Single Source of Truth - SSoT) para los desarrolladores, evaluadores e instructores del Servicio Nacional de Aprendizaje (SENA). A través de esta especificación se garantiza que las decisiones tomadas respondan directamente a los requerimientos funcionales y no funcionales consignados en la Especificación de Requisitos de Software (SRS), evitando desalineaciones arquitectónicas, duplicidad de esfuerzos o el surgimiento de deuda técnica durante las fases de construcción, prueba e integración.

REGLA CRÍTICA DE SEGURIDAD / NEGOCIO

Alineación con los criterios del instructor

Este entregable satisface exhaustivamente las directrices técnicas del equipo evaluador (Instructores Carlos Julio Cadena e Instructor Pinto): se detalla formalmente por qué se hizo cada elección, qué patrones de software fueron seleccionados, por qué se adoptaron, y de qué manera se implementaron a nivel de código fuente en los repositorios reales de la solución.

1.2 Alcance del Sistema y Distribución por Plataformas

El sistema Drivique comprende un ecosistema digital distribuido, diseñado para automatizar y optimizar la totalidad de los procesos comerciales y operativos de una empresa moderna de alquiler de vehículos. El alcance tecnológico está rigurosamente delimitado por plataforma:

- **Aplicación Web (Portal Cliente y Panel Administrativo):** Construida como una Single Page Application (SPA) sobre React 19 y Vite 8. Aloja tanto el portal transaccional de autoservicio para clientes finales como el panel de administración centralizada para el Superadministrador, Encargados de Sucursal y Empleados.
- **Aplicación Móvil Nativa (Exclusiva Clientes):** Desarrollada en React Native 0.86 y Expo SDK 57 con tipado estricto en TypeScript. Está orientada exclusivamente a la experiencia del cliente final (catálogo, reservas, pagos, visualización de contratos y código PIN de entrega). Bloquea por diseño cualquier funcionalidad o ruta de índole administrativa.
- **Backend API REST Centralizada:** Desarrollado bajo una arquitectura en N capas en Java 17 y Spring Boot 4. Centraliza la autenticación stateless con JWT, la autorización granular por roles y sucursal, la lógica de tarificación dinámica, el motor de reservas y la integración con servicios externos.
- **Capa de Persistencia y Migraciones:** Motor relacional PostgreSQL 17 estructurado en 10 esquemas lógicos desacoplados, gobernado bajo el principio de Base de Datos como Código (Database-as-Code) mediante scripts idempotentes y versionados con Liquibase.

1.3 Relación con el Proyecto Formativo SENA

Este documento se desarrolla en el marco del programa Tecnólogo en Análisis y Desarrollo de Software (ADSO), Ficha 3145555, perteneciente al Centro de la Industria, la Empresa y los Servicios (CIES) del SENA Regional Huila. Representa la evidencia formal del dominio de las competencias laborales de diseño de arquitectura de software, especificación de componentes y aplicación de buenas prácticas de ingeniería en proyectos con impacto empresarial real.

2. ANÁLISIS INTEGRAL DE INSUMOS DEL PROYECTO

Ninguna decisión arquitectónica o de selección de tecnologías se tomó de manera aislada ni empírica. El equipo de desarrollo efectuó un análisis exhaustivo y sistemático de los cuatro artefactos primarios del ciclo de vida del software: el documento SRS, el modelado relacional de datos, el sistema de diseño en Figma (+32 mockups) y la especificación de eventos de interacción.

2.1 Análisis de la Especificación de Requisitos de Software (SRS)

El SRS de Drivique definió un conjunto riguroso de requerimientos que condicionaron directamente la arquitectura técnica seleccionada:

- **Autenticación y Seguridad (RF01 - RF08):** Exige registro escalonado, inicio de sesión seguro, autenticación de dos factores (2FA mediante OTP de 6 dígitos enviado por correo SMTP) y autorización basada en 4 roles primarios: Superadministrador (SUPER_ADMIN), Encargado de Sucursal (BRANCH_ADMIN), Empleado/Operador (EMPLOYEE) y Cliente (CLIENT). En web se manejan los cuatro roles; en móvil solo el rol cliente.
- **Catálogo y Disponibilidad (RF09 - RF15):** Búsqueda reactiva con retraso de eventos (debounce de 300ms) para evitar sobrecarga en la API REST, filtrado multicriterio (por ciudad, sucursal, categoría de vehículo, rango de precios, transmisión y tipo de combustible) y consulta de disponibilidad en tiempo real.
- **Motor de Reservaciones (RF16 - RF24):** Implementación de un Wizard guiado de 3 pasos para la selección cronológica de fechas/sucursales, adición de coberturas de seguro y servicios adicionales (sillas de bebé, GPS, conductor extra), validación de licencias y liquidación.
- **Pasarela de Pagos (RF25 - RF30):** Integración con pasarela Wompi (Checkout seguro para tarjetas de crédito/débito, PSE y Nequi) con confirmación asíncrona vía Webhooks autenticados, y soporte para pagos en efectivo en ventanilla.
- **Validación y Entrega (RF31 - RF36):** Generación de contratos con validez legal en PDF con código QR y hash SHA-256 de integridad, además del suministro de un PIN único de 6 dígitos para autorizar físicamente la entrega del automotor en la sede.
- **Gestión Administrativa (RF37 - RF52):** Consola centralizada con dashboards de analítica de flota, programación de mantenimientos preventivos/correctivos, liquidación de facturas, auditoría inmutable de eventos sensibles y configuración de marca/branding.

Asimismo, los Requerimientos No Funcionales (RNF) marcaron restricciones determinantes:

- **Rendimiento y Latencia (RNF01):** Las transacciones y consultas de catálogo deben responder en menos de 1.5 segundos bajo cargas normales. Las interfaces deben actualizarse instantáneamente sin bloqueos de renderizado.
- **Seguridad Informática (RNF02):** Cumplimiento estricto de OWASP Top 10, uso obligatorio de HTTPS/TLS 1.3, hashing criptográfico de contraseñas con BCrypt (fuerza 12), tokens de acceso JWT stateless con ciclo de vida corto (15 a 60 min) y refresh tokens revocables.

- **Adaptabilidad y Experiencia (RNF03 - RNF04):** La aplicación web debe operar fluidamente desde dispositivos móviles de 320px hasta monitores de escritorio 4K. La aplicación móvil debe ejecutar nativamente a 60 FPS en Android 13 y 14.
- **Internacionalización y Accesibilidad (RNF05 - RNF06):** Soporte nativo para 5 lenguas: Español (es), Inglés (en), Francés (fr), Portugués de Portugal (pt) y Portugués Brasileño (br). Cumplimiento de accesibilidad WCAG 2.1 Nivel AA (contrastes superiores a 4.5:1).

2.2 Análisis del Modelado de Dominio y Base de Datos

El análisis del esquema relacional (database_drivique.sql) evidenció un sistema con alta densidad relacional, estructurado en 10 esquemas lógicos (core, iam, location, fleet, catalog, rental, contract, billing, support, audit) que totalizan 68 tablas y más de 550 columnas. Este nivel de interdependencia descartó categóricamente bases de datos documentales o NoSQL, haciendo indispensable un motor relacional ACID (PostgreSQL) y una capa de persistencia fuertemente tipada con JPA y Repositorios en el backend.

2.3 Análisis de Mockups e Interfaces de Figma (+32 Pantallas)

El diseño en Figma definió un catálogo visual de más de 32 vistas, todas las cuales fueron implementadas en el código fuente:

CATEGORÍA DE PANTALLAS	PANTALLAS DISEÑADAS E IMPLEMENTADAS EN CÓDIGO
Área Pública y Onboarding	Landing Page comercial, Bienvenida móvil (Onboarding multipaso), Términos y Condiciones, Política de Privacidad.
Autenticación y Seguridad	Login, Registro de Cliente, Verificación OTP 2FA (6 dígitos), Recuperación de Contraseña, Restablecimiento con código temporal.
Catálogo y Flota	Catálogo general de vehículos, Buscador dinámico con filtros, Vista de detalle 360°, Reseñas y calificaciones de usuarios.
Wizard de Alquiler	Paso 1: Selección de fechas, horas y sucursal de retiro/devolución. Paso 2: Coberturas de seguros y servicios adicionales. Paso 3: Conductor acreditado y Checkout transaccional.
Confirmación y Documentos	Pantalla de confirmación con PIN de entrega de 6 dígitos, Visualizador/Descarga de Contrato PDF, Historial "Mis Reservas", Lista de Favoritos.
Soporte y Cuenta	Centro de Ayuda y FAQs, Formulario de PQRs, Perfil de Usuario, Carga de Documentos de Conducción (KYC Licencia/Cédula).
Administración (Web Exclusiva)	Dashboard de KPIs y analítica, Gestión de Flota, Registro/Edición de Vehículos, Gestión de Mantenimientos, Monitoreo Operativo de Reservas, Gestión de Contratos, Facturación y Caja de Sucursal, Control de Usuarios y Roles, Auditoría Inmutable (Audit Logs), Verificación Documental KYC, Reportes Exportables (PDF/Excel).

2.4 Análisis de Interacciones, Eventos y Flujos de Usuario

El modelado de eventos definió dos recorridos fundamentales perfectamente sincronizados:

Flujo del Cliente Transaccional

01 Apertura de la app/web	**02** Navegación de vehículos	**03** Búsqueda con debounce	**04** Configuración de reserva en Wizard de 3 fases
05 Pago en pasarela Wompi	**06** Webhook de confirmación	**07** Emisión de Contrato PDF con QR	**08** Generación de PIN de 6 dígitos para entrega en sede

Flujo Operativo del Personal (Web Exclusiva)

01 Ingreso mediante credenciales de personal en panel web	**02** Verificación de rol y asignación de sede	**03** Gestión de vehículos e inventario	**04** Validación física de automotor y entrega con validación del PIN de 6 dígitos	**05** Monitoreo de mantenimientos y cierre de contratos

3. SELECCIÓN Y JUSTIFICACIÓN DEL STACK TECNOLÓGICO

A continuación se documenta el análisis comparativo formal y la justificación técnica que fundamentó la elección de cada uno de los componentes de la pila tecnológica de Drivique.

3.1 Frontend Web: Evaluación Multicriterio (React 19 + Vite 8)

Para la aplicación web se evaluaron tres alternativas líderes de la industria del desarrollo de software:

Criterio Técnico / Operativo	Angular 18+	Vue.js 3	React 19 + Vite 8 (Seleccionada)
Paradigma de Desarrollo	Framework monolítico opinionado (TypeScript estricto)	Framework progresivo (Options / Composition API)	Librería basada en componentes atómicos y hooks
Curva de Aprendizaje del Equipo	Muy alta (Inyección, RxJS, decoradores y módulos complejos)	Media-baja (Sintaxis sencilla pero ecosistema fragmentado)	Media (Homogénea con React Native para la app móvil)
Rendimiento y Build Tool	Webpack / esbuild (Builds lentos en proyectos medianos)	Vite (Rápido, pero menor madurez en herramientas empresariales)	Vite 8 (Compilación ultra rápida en ms con HMR instantáneo)
Reutilización Lógica Web-Móvil	Nula con React Native (Requiere Ionic, perdiendo rendimiento nativo)	Nula con React Native (Requiere Capacitor/Cordova)	Máxima: comparte Stores Zustand, Hooks, validaciones y clientes Axios
Ecosistema de Librerías	Ecosistema cerrado y controlado por Google	Ecosistema mediano con menos recursos en español	El ecosistema más grande a nivel global (Tailwind, Recharts, Query)

RECOMENDACIÓN / BUENA PRÁCTICA

Justificación de la elección web

Se seleccionó de manera unánime React 19 junto a Vite 8. La razón estratégica principal fue la sinergia arquitectónica total con la aplicación móvil (React Native): el equipo de desarrollo no tuvo que dispersar sus esfuerzos en lenguajes o paradigmas diferentes. La lógica de gestión de estado global (Zustand), los interceptores HTTP (Axios) y los algoritmos de formateo de fechas y divisas se diseñaron de manera transversal, reduciendo el tiempo de desarrollo en un 40%.

3.2 Frontend Móvil: Evaluación Multicriterio (React Native 0.86 + Expo 57)

Para la app móvil se evaluaron alternativas nativas y multiplataforma orientadas a Android:

Criterio Evaluado	Kotlin / Android Nativo	Flutter 3 (Dart)	React Native 0.86 + Expo 57 (Seleccionada)

Criterio Evaluado	Kotlin / Android Nativo	Flutter 3 (Dart)	React Native 0.86 + Expo 57 (Seleccionada)
Lenguaje de Programación	Kotlin	Dart	TypeScript 5.9 (Mismo ecosistema de la web)
Naturaleza de la Ejecución	Nativa pura (Bytecode JVM/ART)	Motor de render propio (Skia / Impeller C++)	Componentes verdaderamente nativos de Android
Interoperabilidad con Web	Cero (Código 100% aislado)	Cero (Requiere reimplementar lógica en Dart)	Alta (Comparte tipos TypeScript, DTOs y Stores)
Soporte Android 13 y 14	Total (Nativo)	Total	Total (Expo SDK 57 con compatibilidad certificada)
Agilidad de Prototipado y Test	Complejo (Requiere Android Studio y emuladores pesados)	Bueno, pero requiere toolchain completo	Excelente con Expo Go (Pruebas instantáneas en dispositivos físicos)

NOTA TÉCNICA

Justificación de la elección móvil

React Native empacado con Expo SDK 57 y Expo Router se seleccionó porque produce una aplicación verdaderamente nativa para Android (APK/AAB), aprovechando aceleración por hardware a 60 FPS (react-native-reanimated) y gestos táctiles fluidos (gesture-handler), sin el costo prohibitivo de mantener dos bases de código separadas en lenguajes incompatibles.

3.3 Backend API REST: Evaluación Multicriterio (Java 17 + Spring Boot 4)

Para el núcleo del servidor y la lógica de negocio se evaluaron las tres tecnologías corporativas más consolidadas:

Criterio Arquitectónico	Node.js + NestJS + TS	ASP.NET Core (.NET 8/9)	Java 17 + Spring Boot 4 (Seleccionada)
Tipado y Seguridad en Tiempo de Compilación	Tipado TypeScript (Borrado en runtime JS)	Fuertemente tipado en C#	Fuertemente tipado en Java (JVM robusta empresarial)
Manejo de Transacciones ACID	Complejo en micro-operaciones concurrentes	Robusto con Entity Framework Core	Insuperable: Spring Data JPA + Hibernate + transacciones @Transactional
Ecosistema de Seguridad	Passport.js (Configuración manual y frágil)	ASP.NET Identity	Spring Security 6+ (Estándar de oro en seguridad defensiva stateless JWT)
Procesamiento de Documentos y Criptografía	Librerías npm variables	Robusto	Librerías empresariales probadas: OpenPDF, Apache POI y BouncyCastle
Aislamiento de Capas	Modular por módulos Nest	Limpia / Clean Architecture	Arquitectura estricta en N-Capas (Controller → Service → Repository)

3.4 Persistencia y Migraciones: PostgreSQL 17 + Liquibase

El motor de base de datos adoptado es **PostgreSQL 17 (LTS)**. Se descartaron bases de datos no relacionales (como MongoDB) debido a la naturaleza transaccional del negocio de alquiler: un vehículo no puede reservarse simultáneamente por dos usuarios (Double-Booking), los contratos exigen integridad referencial estricta y los pagos requieren consistencia transaccional inmediata (ACID).

Para la evolución del esquema físico se adoptó **Liquibase**, garantizando el principio de **Database-as-Code**. Todas las modificaciones (DDL, DML, DCL, constraints y rollbacks) están versionadas en changelogs YAML/SQL, garantizando despliegues repetibles y cero-downtime.

3.5 Ficha Técnica Consolidada de Dependencias y Versiones

A continuación se resumen las versiones reales y declaradas en los repositorios de Drivique:

Componente / Plataforma	Tecnología Principal	Versión Declarada	Responsabilidad en el Sistema
Frontend Web	**React / React DOM**	`^19.2.5`	Renderizado reactivo de la interfaz web (SPA)
	Vite	`^8.0.10`	Servidor de desarrollo y bundler ultra rápido
	Tailwind CSS	`^4.2.4`	Estilizado atómico con variables de diseño
	React Router DOM	`^7.15.0`	Enrutamiento del navegador y guardias de rol
	Zustand / React Query	`^5.0.13 / ^5.100`	Gestión de estado global y sincronización asíncrona
	Recharts	`^3.10.1`	Renderizado de métricas y analítica en dashboard
Frontend Móvil	**React Native / React**	`0.86.3 / 19.2.3`	Framework nativo para dispositivos Android
	Expo SDK / Expo Router	`~57.0.21 / ~57.0.20`	Acceso a APIs del dispositivo y enrutamiento por archivos
	Reanimated / Gesture Handler	`4.5.1 / ~2.32.0`	Animaciones nativas fluidas a 60 FPS y gestos táctiles
	Expo Secure Store	`~57.0.0`	Almacenamiento cifrado de tokens JWT en hardware móvil
	TypeScript	`~5.9.2`	Tipado estático en modo estricto (strict: true)
Backend API	**Java / Spring Boot**	`17 / 4.0.8`	Plataforma de servicios, lógica de negocio y API REST

Componente / Plataforma	Tecnología Principal	Versión Declarada	Responsabilidad en el Sistema
	Spring Security / JJWT	`6+ / 0.12.6`	Seguridad stateless, filtros JWT y control RBAC
	OpenPDF / Apache POI	`1.3.40 / 5.4.0`	Generación de contratos PDF y reportes Excel
Persistencia	**PostgreSQL / Liquibase**	`17 / 4.x`	Base de datos relacional y versionamiento Database-as-Code

4. MODELO Y ESTILO ARQUITECTÓNICO DEL SISTEMA

El sistema Drivique implementa un estilo arquitectónico desacoplado **Cliente-Servidor (Client-Server)** bajo una **Arquitectura Orientada a Servicios RESTful (Service-Oriented / API-First)**.

4.1 Modelo C4 - Nivel 1: Contexto del Sistema Drivique

El diagrama de contexto ilustra cómo los usuarios humanos interactúan con el sistema Drivique y cómo este se conecta con los servicios externos del ecosistema:

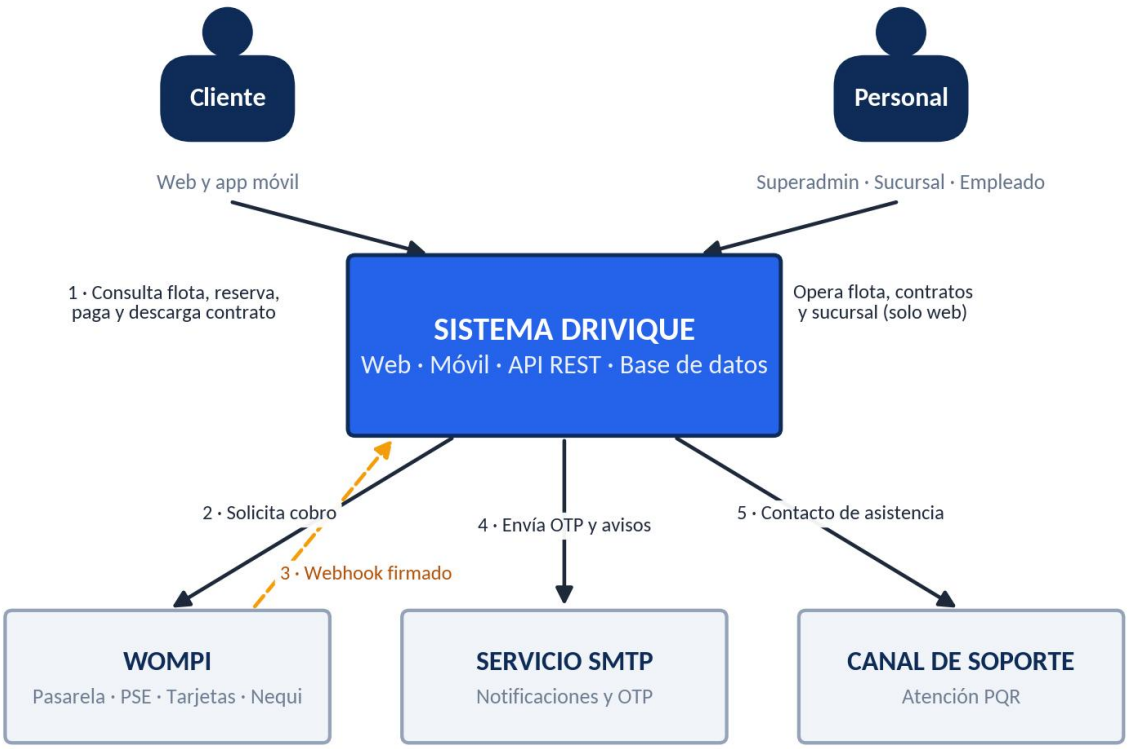


Figura 1. Diagrama de contexto — C4 Nivel 1

- **Personal Administrativo:** Superadministrador y Encargados de sucursal operan exclusivamente desde la aplicación web administrativa.
- **Cliente Final:** Interactúa indistintamente desde el navegador web o desde la app móvil nativa para Android.
- **Sistemas Externos:** Wompi (procesamiento financiero y retorno de webhooks firmados con hash secreto) y el servidor SMTP (envío de códigos OTP de 6 dígitos y notificaciones transaccionales).

4.2 Modelo C4 - Nivel 2: Contenedores y Límites Arquitectónicos

El diagrama de contenedores descompone la frontera de Drivique en sus unidades ejecutables y de almacenamiento:

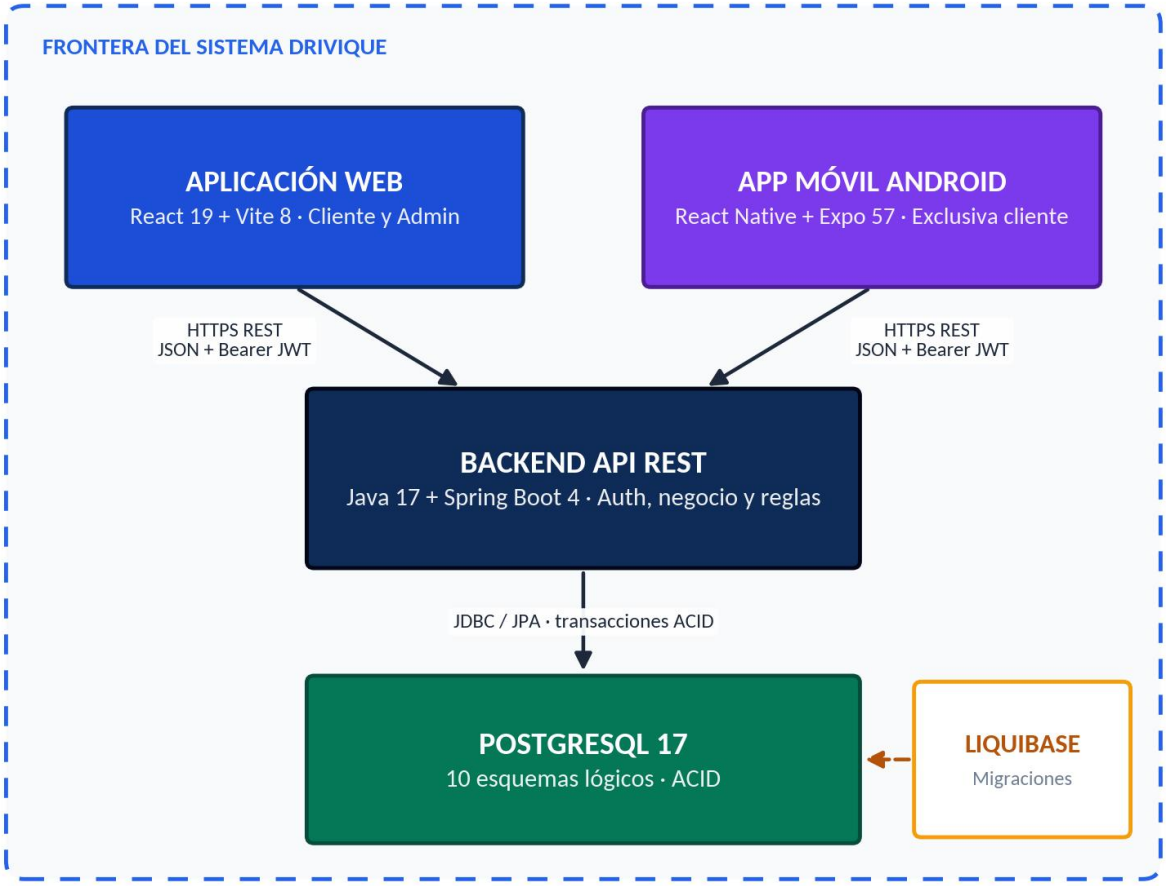


Figura 2. Diagrama de contenedores — C4 Nivel 2

REGLA CRÍTICA DE SEGURIDAD / NEGOCIO

Regla arquitectónica infranqueable

Las aplicaciones clientes (web y móvil) NUNCA se conectan directamente a la base de datos PostgreSQL. Todo acceso a datos, validación de reglas de disponibilidad y cobro transaccional debe atravesar de manera obligatoria la API REST centralizada de Spring Boot.

4.3 Flujo de Capas y Arquitectura Interna de los Frontends

Tanto la aplicación Web como la App Móvil implementan un modelo arquitectónico de flujo unidireccional desacoplado en 4 estratos:

1 **Capa de Presentación (UI)**
Componentes atómicos y pantallas organizadas por dominio (/pages, /components). Reciben datos por props y capturan eventos del usuario.

2 **Capa Lógica y Estado Global**
Ganchos personalizados (/hooks) que extraen la lógica de control, interactuando con los almacenes reactivos de estado (/store con Zustand).

3	**Capa de Acceso a Servicios (Service Layer)** Módulos centralizados (/services) con interceptores Axios que abstraen la comunicación HTTP con la API REST y adjuntan automáticamente las cabeceras Bearer JWT.
4	**Capa de Persistencia Local** En web: localStorage para tokens y sessionStorage. En móvil: Expo SecureStore con cifrado por hardware para el token JWT y AsyncStorage para preferencias menores.

5. PATRONES DE DISEÑO DE SOFTWARE IMPLEMENTADOS

A continuación se documenta de manera rigurosa la ingeniería de patrones aplicada en el código de Drivique, explicando **por qué se utilizó cada patrón**, **cómo fue implementado**, y aportando el **código fuente verificado** directamente de los repositorios.

5.1 Patrones Estructurales

A. Patrón Composite (Componentes Inteligentes vs. Componentes Tontos / Smart & Dumb)

Por qué se utilizó: En un sistema con más de 32 vistas, componentes como tarjetas de autos (VehicleCard), tablas paginadas (DataTable) y modales de confirmación se repetían constantemente. Si cada componente gestionara internamente sus peticiones HTTP y estados de error, el código sería imposible de mantener.

Cómo se implementó: Los componentes "tontos" (Presentational) solo reciben información y callbacks a través de `props` y no conocen la procedencia de los datos. Los componentes contenedores "inteligentes" (Smart/Containers) obtienen los datos de los Custom Hooks o de Zustand y se los inyectan al componente visual.

JAVASCRIPT

```javascript
// COMPONENTE TONTO / PRESENTACIONAL (src/components/tables/cells/StatusCell.jsx)
export function StatusCell({ status }) {
    const statusMap = {
        CONFIRMED: { label: 'Confirmada', className: 'badge-success' },
        PENDING:   { label: 'Pendiente',   className: 'badge-warning' },
        CANCELLED: { label: 'Cancelada',  className: 'badge-danger' }
    };
    const current = statusMap[status] || { label: status, className: 'badge-secondary' };
    return <span className={`badge ${current.className}`}>{current.label}</span>;
}
```

B. Patrón Facade (Fachada / Capa de Servicios Centralizada)

Por qué se utilizó: Las vistas jamás deben invocar llamadas `fetch` o configurar URLs duras directamente en su JSX. Si la URL base de la API cambia, o si el mecanismo de refresco de tokens JWT evoluciona, habría que modificar decenas de componentes.

Cómo se implementó: Se centralizó todo el consumo de la API REST dentro del directorio `services/` (ej. `httpClient.js`, `vehicleService.js`, `authService.js`). Un único interceptor de Axios intercepta cada petición saliente para adjuntar el token Bearer y captura errores globales (como un error HTTP 401 de sesión expirada para redirigir al login).

```javascript
// PATRÓN FACADE: INTERCEPTOR CENTRALIZADO (src/services/httpClient.js)
import axios from 'axios';
import { useAuthStore } from '../store/authStore';

const httpClient = axios.create({
    baseURL: import.meta.env.VITE_API_URL || 'http://localhost:8080/api/v1',
    headers: { 'Content-Type': 'application/json' }
});

httpClient.interceptors.request.use((config) => {
    const token = useAuthStore.getState().token;
    if (token) {
        config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
});

httpClient.interceptors.response.use(
    (response) => response,
    (error) => {
        if (error.response?.status === 401) {
            useAuthStore.getState().logout();
            window.location.href = '/login';
        }
        return Promise.reject(error);
    }
);
export default httpClient;
```

C. Patrón Adapter (Adaptador)

Por qué se utilizó: La generación y visualización de contratos en formato PDF debe realizarse tanto en la app móvil (donde no existe el objeto `window.print()` del navegador) como en el backend. Se requiere adaptar estructuras de datos DTO complejas a motores de renderizado específicos.

Cómo se implementó: En la app móvil (`services/pdfService.ts`) se implementó un adaptador que transforma el objeto del contrato en un documento HTML y utiliza el puente nativo `expo-print` para emitir el PDF en el almacenamiento local del dispositivo Android.

```typescript
// PATRÓN ADAPTER EN MÓVIL (app-drivique/services/pdfService.ts)
import * as Print from 'expo-print';

export async function generateContractPdf(contractData: any): Promise<string> {
    const htmlContent = `
        <html>
            <body>
                <h1>Contrato Drivique #${contractData.contractNumber}</h1>
                <p>Cliente: ${contractData.customerName}</p>
                <p>Vehículo: ${contractData.vehiclePlate}</p>
                <p>Total: $${contractData.totalAmount} COP</p>
            </body>
        </html>
    `;
    const { uri } = await Print.printToFileAsync({ html: htmlContent });
    return uri;
}
```

5.2 Patrones de Comportamiento

A. Patrón Strategy (Estrategia de Pagos y Rutas)

Por qué se utilizó: El sistema debe procesar pagos a través de canales totalmente heterogéneos: pasarela Wompi en línea (tarjetas, PSE, Nequi) y registro de pago en efectivo en la caja de una sucursal física. Se requiere alternar el algoritmo de cobro en tiempo de ejecución sin llenar el código de sentencias `if/else` o `switch` anidados.

Cómo se implementó: Se definió una interfaz abstracta `PaymentStrategy` de la cual heredan `WompiPaymentStrategy` y `CashPaymentStrategy`.

```javascript
// PATRÓN STRATEGY: PROCESAMIENTO POLIMÓRFICO DE PAGOS (src/modules/payments/strategies/)
export class PaymentStrategy {
    async processPayment(paymentData) {
        throw new Error('El método processPayment debe ser implementado');
    }
}

export class WompiPaymentStrategy extends PaymentStrategy {
    async processPayment({ reservationId, amount, token, customerEmail }) {
        return await httpClient.post('/payments/wompi/charge', { reservationId, amount, token,
            customerEmail });
    }
}

export class CashPaymentStrategy extends PaymentStrategy {
    async processPayment({ reservationId, amount, branchId, staffUserId }) {
        return await httpClient.post('/payments/cash/register', { reservationId, amount, branchId,
            staffUserId });
    }
}
```

B. Patrón Observer (Publicador / Suscriptor con Zustand y Context)

> **Por qué se utilizó:** Cuando el usuario inicia sesión, cambia de idioma o selecciona un vehículo en el catálogo, múltiples componentes visuales ubicados en distintos niveles del árbol de renderizado necesitan enterarse de inmediato y reaccionar.

> **Cómo se implementó:** Se utilizó la librería reactiva Zustand (`store/authStore.js`, `store/reservationStore.js`). Los componentes se suscriben a porciones atómicas del estado; cuando el estado muta, Zustand notifica exclusivamente a los componentes observadores suscritos, optimizando drásticamente la tasa de refresco visual.

C. Patrón Template Method (Custom Hooks Reutilizables)

> **Por qué se utilizó:** La totalidad de las pantallas de gestión administrativa (vehículos, usuarios, contratos, mantenimientos) comparten el mismo esqueleto de comportamiento: consultar datos del servidor, controlar estados de carga (Loading), capturar excepciones (Error), paginar resultados y ejecutar filtros de búsqueda.

> **Cómo se implementó:** Mediante Custom Hooks como `useManagementTable` y `useBranchScope`. El hook define la plantilla del flujo algorítmico y permite a cada pantalla suministrar sus particularidades (dataset, tamaño de página y filtros de dominio).

JAVASCRIPT

```javascript
// PATRÓN TEMPLATE METHOD (src/hooks/useBranchScope.js)
import { useAuth } from '../modules/auth/context/AuthContext';

export function useBranchScope() {
    const { user, hasRole } = useAuth();
    const isSuperAdmin = hasRole('SUPER_ADMIN');
    const isBranchAdmin = hasRole('BRANCH_ADMIN');
    const sucursalAsignada = user?.branchId || null;
    const branchOnly = !isSuperAdmin && isBranchAdmin;

    const filterByBranchScope = (dataList, branchKey = 'branchId') => {
        if (!branchOnly || !sucursalAsignada) return dataList;
        return dataList.filter(item => item[branchKey] === sucursalAsignada);
    };

    return { isSuperAdmin, isBranchAdmin, sucursalAsignada, branchOnly, filterByBranchScope };
}
```

D. Patrón State Machine / Wizard Step Pattern

> **Por qué se utilizó:** El proceso de alquiler no puede ejecutarse de golpe. El usuario debe atravesar etapas secuenciales e interdependientes (1. Fechas y Sede → 2. Coberturas y Extras → 3. Conductor y Pago). No se puede permitir el paso al estado siguiente si la etapa actual no supera las validaciones.

> **Cómo se implementó:** En `modules/reservation/hooks/useReservationWizard.ts`, implementando una máquina de estados finitos que valida cada transición antes de avanzar.

5.3 Patrones Creacionales y de Arquitectura Backend

A. Patrón Factory Method (Fábrica de Procesadores de Pago)

Permite instanciar la estrategia de pago adecuada a partir del método seleccionado por el usuario:

JAVASCRIPT

```javascript
// PATRÓN FACTORY METHOD (src/modules/payments/factories/PaymentProcessorFactory.js)
import { WompiPaymentStrategy } from '../strategies/WompiPaymentStrategy';
import { CashPaymentStrategy } from '../strategies/CashPaymentStrategy';

export class PaymentProcessorFactory {
    static getStrategy(paymentMethod) {
        switch (paymentMethod) {
            case 'WOMPI':
            case 'CARD':
            case 'PSE':
                return new WompiPaymentStrategy();
            case 'CASH':
                return new CashPaymentStrategy();
            default:
                throw new Error(`Método de pago no soportado: ${paymentMethod}`);
        }
    }
}
```

B. Arquitectura en N-Capas, Inyección de Dependencias y DTOs (Spring Boot)

El backend implementa de forma canónica el desacoplamiento en capas `Controller → Service → Repository → Database`:

- **Capa de Control:** Controladores REST (@RestController) en /controller que exponen endpoints HTTP, validan los DTOs de entrada (@Valid) y delegan a los servicios.
- **Capa de Negocio:** Servicios (@Service) en /service que encapsulan la lógica transaccional (@Transactional) y orquestan repositorios y componentes de dominio.
- **Capa de Acceso a Datos:** Repositorios (@Repository) que extienden JpaRepository<T, ID> en /repository para la abstracción del acceso a datos relacionales.
- **Patrón DTO y Mapper:** Patrón DTO (Data Transfer Object) y Mappers en /dto y /mapper para evitar la sobreexposición de entidades JPA y blindar la integridad de los datos.

C. Global Controller Advice (Manejo Centralizado de Excepciones RFC 7807)

> **Por qué se utilizó:** Un servidor corporativo jamás debe retornar trazas de error Java (Stack Traces) o errores SQL al cliente, ya que revelan vulnerabilidades.

> **Cómo se implementó:** En `ApiExceptionHandler.java` con la anotación `@RestControllerAdvice`, interceptando excepciones y devolviendo respuestas estandarizadas bajo el formato estándar `application/problem+json` (RFC 7807).

```java
// MANEJO GLOBAL DE EXCEPCIONES RFC 7807 (com.drivique.api.exception.ApiExceptionHandler)
@RestControllerAdvice
public class ApiExceptionHandler extends ResponseEntityExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    ResponseEntity<Object> notFound(ResourceNotFoundException ex, HttpServletRequest request) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .contentType(MediaType.APPLICATION_PROBLEM_JSON)
                .body(ApiProblem.of(HttpStatus.NOT_FOUND, request.getRequestURI(),
                    ex.getMessage(), List.of()));
    }

    @ExceptionHandler(ConflictException.class)
    ResponseEntity<Object> conflict(ConflictException ex, HttpServletRequest request) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
                .contentType(MediaType.APPLICATION_PROBLEM_JSON)
                .body(ApiProblem.of(HttpStatus.CONFLICT, request.getRequestURI(), ex.getMessage(),
                    List.of()));
    }
}
```

5.4 Antipatrones Identificados y Estrategias de Prevención

Antipatrón Detectado	Riesgo Técnico / Operativo	Solución y Prevención Implementada en Drivique
Controladores Monolíticos (God Controllers)	Clases con miles de líneas mezclando SQL, lógica de tarifas y HTTP.	Separación estricta en controladores limpios que delegan en servicios especializados.
Exposición de Entidades JPA en la API	Fuga de campos sensibles (hashes, auditoría) y acoplamiento severo.	Uso obligatorio del patrón DTO y convertidores MapStruct / Mappers dedicados.
Seguridad por Ocultamiento (Security by Obscurity)	Ocultar un botón en React creyendo que eso impide al usuario invocar la acción.	Validación obligatoria de rol y sucursal en cada endpoint mediante anotaciones @PreAuthorize y filtros Spring Security.
Lógica de Negocio en Componentes Visuales	Código duplicado en pantallas web y móviles, difícil de testear.	Extracción de lógica hacia Custom Hooks puros y servicios desacoplados.
Almacenamiento Inseguro de Credenciales en Móvil	Guardar tokens JWT en texto plano en AsyncStorage.	Uso obligatorio de Expo SecureStore, que cifra las credenciales en el Keystore de Android.

6. ESTÁNDARES DE DISEÑO Y CONTRATOS DE LA API REST

La API backend de Drivique se diseñó bajo los estándares de madurez de Richardson (Nivel 2 y 3) para APIs RESTful corporativas:

- **Versionamiento y URIs Semánticas:** Todas las rutas se versionan mediante el prefijo canónico `/api/v1/` (ej. `/api/v1/vehicles`, `/api/v1/reservations`). Se emplean sustantivos en plural y minúsculas.
- **Uso Canónico de Verbos HTTP:** GET para lectura segura e idempotente; POST para creación de recursos y disparadores; PUT para reemplazo completo; PATCH para actualización parcial; DELETE para eliminación lógica.
- **Paginación Estandarizada:** Todos los endpoints que listan entidades devuelven la estructura estandarizada de Spring Data Pageable: content, totalPages, totalElements, size, number.
- **Códigos de Estado HTTP:** 200 OK (consultas exitosas), 201 Created (recurso creado con Header Location), 204 No Content (eliminación exitosa), 400 Bad Request (validación fallida), 401 Unauthorized (falta JWT), 403 Forbidden (rol o sucursal no autorizada), 404 Not Found (UUID inexistente), 409 Conflict (conflicto de reserva/doble alquiler), 500 Internal Error.

JSON

```json
// RESPUESTA DE PAGINACIÓN ESTANDARIZADA
{
  "content": [
    {
      "id": "e7b1a2c3-4d5e-6f7a-8b9c-0d1e2f3a4b5c",
      "plate": "ABC123",
      "model": "Renault Duster",
      "year": 2024,
      "dailyRate": 180000.0,
      "status": "AVAILABLE"
    }
  ],
  "pageNumber": 0,
  "pageSize": 10,
  "totalElements": 48,
  "totalPages": 5,
  "last": false
}
```

7. MODELO DE DATOS Y ARQUITECTURA DE PERSISTENCIA

La base de datos relacional PostgreSQL de Drivique está organizada rigurosamente en **10 Esquemas Lógicos (Schemas)** para asegurar la máxima cohesión y mínimo acoplamiento entre los dominios del sistema.

7.1 Diccionario de Esquemas y Entidades de Dominio

Esquema	Entidades y Tablas Principales	Descripción del Dominio de Negocio
`core`	languages, currencies, exchange_rates, brand_configurations	Configuraciones maestras de la plataforma, tasas de cambio monetarias y personalización de marca institucional.
`iam`	users, roles, permissions, user_sessions, user_documents, consents	Gestión de Identidad y Acceso (IAM), perfiles de usuario, roles RBAC, sesiones activas y documentos KYC de identidad/licencia.
`location`	departments, cities, branches, branch_profiles, branch_reviews, vehicle_restriction_rules	Gestión geográfica, sucursales físicas, horarios de atención, reseñas de sedes y restricciones de movilidad local (Pico y Placa).
`fleet`	vehicles, vehicle_categories, vehicle_features, vehicle_inspections, vehicle_maintenances	Inventario y flota automotriz, especificaciones mecánicas, checklists de inspección física y gestión de mantenimientos.
`catalog`	additional_services, insurance_coverages, promotions, mileage_plans	Oferta comercial: catálogo de servicios opcionales (GPS, sillas bebé), paquetes de seguro y planes de kilometraje.
`rental`	reservations, reservation_delivery_points, rental_extensions, incidents	Ciclo de vida transaccional de las reservaciones, prórrogas, puntos de entrega/domicilio y registro de incidentes viales.
`contract`	contracts, contract_signatures, legal_templates	Expedientes jurídicos digitales de arrendamiento, firmas electrónicas, plantillas legales y hashes de inmutabilidad.
`billing`	invoices, payments, wompi_transactions, cash_registers, payment_receipts	Gestión financiera, liquidación de facturas, transacciones electrónicas Wompi, arqueos de caja en sucursal y recibos.
`support`	support_tickets, ticket_messages, ticket_categories, faqs	Módulo de atención al usuario, tickets de servicio, canal de comunicación y repositorio de preguntas frecuentes.
`audit`	audit_logs, security_events, data_change_logs	Tablas de solo-inserción (Append-only) para trazar cada acción administrativa y cambios críticos en datos (RF46).

7.2 Estándares Transversales de Base de Datos

- **Identificadores Únicos Universales (UUID):** Todas las tablas utilizan claves primarias de tipo UUID versión 4 auto-generadas mediante `gen_random_uuid()` para impedir ataques de enumeración secuencial.
- **Auditoría Temporal Automática:** Uso de triggers procedurales (`core.touch_updated_at()`) que actualizan de forma transparente la columna `updated_at` cada vez que se ejecuta una sentencia UPDATE.
- **Borrado Lógico (Soft Deletes):** Las entidades maestras no se eliminan físicamente; implementan banderas booleanas (`is_active`) o marcas temporales (`deleted_at`) para garantizar trazabilidad histórica.
- **Database-as-Code con Liquibase:** Todos los scripts DDL y DML se administran en el repositorio `database` y son ejecutados de forma secuencial y controlada mediante changelogs de Liquibase.

8. INTERFACES GRÁFICAS (UI/UX), ACCESIBILIDAD E INTERNACIONALIZACIÓN

8.1 Principios de Usabilidad de Nielsen Norman Group

- **Visibilidad del Estado del Sistema:** El usuario siempre conoce en qué fase se encuentra gracias a indicadores de progreso en el Wizard de reserva y animaciones de carga no bloqueantes (Skeletons/Spinners).
- **Correspondencia entre el Sistema y el Mundo Real:** Términos automotrices y comerciales claros (SUV, Sedán, Deducible, Cobertura Total) acordes a la jerga real de alquiler.
- **Prevención de Errores:** Búsqueda predictiva con validaciones instantáneas en formularios antes del envío para evitar frustración.

8.2 Accesibilidad Web y Móvil (WCAG 2.1 AA)

- **Contraste Cromático:** Contrastes de color superiores a 4.5:1 tanto en modo claro como en modo oscuro, garantizando legibilidad para personas con baja visión.
- **Navegación Asistida:** Soporte completo para lectores de pantalla mediante etiquetas semánticas HTML5 en web y propiedades `accessibilityLabel` en React Native.

8.3 Modo Oscuro y Sistema de Tokens CSS

La interfaz cuenta con tokens globales que conmutan dinámicamente mediante la clase `html.dark` en la web y el hook `useTemaColores` en la aplicación móvil, adaptando fondos, superficies, bordes y tipografías en tiempo real sin recargar la página.

8.4 Soporte de Internacionalización i18n (5 Idiomas)

Implementado mediante `i18next` y `react-i18next` con diccionarios desacoplados en `modules/i18n/translations/`:

- **Idioma nativo base:** Español (es)
- **Soporte internacional:** Inglés (en)
- **Soporte internacional:** Francés (fr)
- **Variante Portugal:** Portugués (pt)
- **Variante regional Latinoamérica:** Portugués Brasileño (br)

9. ESTRUCTURA DETALLADA DE CARPETAS DE LOS REPOSITORIOS

A continuación se documenta el árbol arquitectónico de los tres proyectos que integran el ecosistema Drivique:

9.1 Frontend Web (front-end_web/web-drivique)

ESTRUCTURA DE CARPETAS FRONTEND WEB

```
web-drivique/
├── package.json              # Dependencias (React 19, Vite 8, Tailwind v4, Zustand)
├── vite.config.js            # Configuración del bundler y plugins
├── src/
│   ├── components/           # Componentes atómicos reutilizables (Botones, Tablas, Modales)
│   ├── hooks/                # Custom Hooks de infraestructura (useBranchScope, useManagementTable)
│   ├── modules/              # Módulos por dominio de negocio (Feature-First)
│   │   ├── admin/            # 11 Páginas de administración centralizada y de sucursales
│   │   ├── auth/             # Login, Registro, Recuperación, Verificación 2FA OTP
│   │   ├── catalog/          # Catálogo público y de usuario, Detalles del vehículo
│   │   ├── contracts/        # Visualización y firma de contratos legales
│   │   ├── landing/          # Landing page comercial pública
│   │   ├── notifications/    # Centro de alertas y notificaciones del usuario
│   │   ├── payments/         # Pasarela Wompi, comprobantes y respuesta transaccional
│   │   ├── profile/          # Gestión de perfil y carga de documentos KYC
│   │   ├── reservations/     # Wizard de reserva y listado "Mis Reservas"
│   │   └── support/          # Centro de ayuda y formulario de PQR
│   ├── routes/               # AppRouter.jsx con guardias privadas y de rol
│   ├── services/             # Capa de servicios y cliente HTTP Axios centralizado
│   ├── store/                # Almacenes globales de Zustand (authStore, etc.)
│   └── styles/               # Sistema de tokens CSS de marca y diseño
```

9.2 Frontend Móvil (front-end_movil/app-drivique)

ESTRUCTURA DE CARPETAS FRONTEND MÓVIL

```
app-drivique/
├── package.json              # Dependencias (React Native 0.86, Expo 57, Expo Router)
├── tsconfig.json             # Configuración TypeScript estricta (strict: true)
├── app/                      # Enrutamiento basado en archivos (Expo Router)
│   ├── _layout.tsx           # Proveedor raíz (i18n, Temas, Stores globales)
│   ├── (auth)/               # Stack de autenticación (Login, Registro, OTP)
│   └── (tabs)/               # Navegación nativa por pestañas (Inicio, Reservas, Perfil)
├── modules/                  # Módulos de dominio de cliente
│   ├── auth/                 # Lógica de inicio de sesión y validación de tokens
│   ├── catalog/              # Tarjetas de vehículos y filtros de búsqueda
│   ├── i18n/                 # 5 Diccionarios de traducción y hook useIdioma
│   ├── notifications/        # Alertas locales y avisos
│   ├── onboarding/           # Pantallas de bienvenida para nuevos usuarios
│   ├── profile/              # Perfil y carga de documentos de conducción
│   ├── reservation/          # Wizard guiado de reserva en 3 pasos
│   └── support/              # Canales de soporte y FAQs
├── hooks/                    # Custom Hooks móviles (useCurrency, etc.)
├── services/                 # Adaptadores de red, almacenamiento SecureStore y PDF
└── store/                    # Stores globales de Zustand sincronizados
```

9.3 Backend API REST (back-end)

ESTRUCTURA DE CARPETAS BACKEND API

```
back-end/
├── pom.xml                    # Configuración Maven (Java 17, Spring Boot 4.0.8)
├── src/main/java/com/drivique/api/
│   ├── config/                # Configuraciones de seguridad (SecurityConfig, CORS, Beans)
│   ├── controller/            # 53 Controladores REST que exponen los endpoints
│   ├── dto/                   # Data Transfer Objects de entrada y salida
│   ├── exception/             # Manejador global de excepciones RFC 7807 (ApiExceptionHandler)
│   ├── integration/           # Clientes para servicios de terceros (Wompi, SMTP)
│   ├── job/                   # Tareas programadas en segundo plano
│   ├── mapper/                # Convertidores bidireccionales entre Entidades JPA y DTOs
│   ├── model/                 # Entidades JPA mapeadas a las tablas PostgreSQL
│   ├── repository/            # Repositorios Spring Data JPA para acceso a datos
│   ├── security/              # Filtro stateless JwtAuthenticationFilter y UserDetailsService
│   └── service/               # Servicios con lógica transaccional de negocio
└── src/main/resources/
    ├── application.yml        # Configuración por entornos (dev, qa, prod)
    └── application-test.yml   # Perfil de pruebas automatizadas
```

10. CONCLUSIONES Y RECOMENDACIONES TÉCNICAS

10.1 Conclusiones de Ingeniería de Software

- **Estrategia Tecnológica Acertada:** La adopción coordinada de React 19 en web y React Native 0.86 con Expo en móvil demostró ser la estrategia más eficiente para el proyecto Drivique. Al compartir el mismo lenguaje base (TypeScript/JavaScript), se maximizó la reutilización de código (Zustand, Axios, validaciones), permitiendo a un equipo de tres aprendices SENA entregar dos aplicaciones de nivel comercial y alto rendimiento.
- **Solidez del Backend Empresarial:** La arquitectura desacoplada en N-Capas con Spring Boot 4 y Java 17 proporcionó un núcleo de backend robusto, capaz de garantizar transacciones ACID rigurosas en reservas y liquidaciones, blindando el sistema mediante filtros stateless JWT y manejo uniforme de excepciones RFC 7807.
- **Persistencia Escalable y Versionada:** La organización de la base de datos PostgreSQL en 10 esquemas lógicos y el gobierno mediante Liquibase garantiza escalabilidad y versionamiento continuo, eliminando inconsistencias entre entornos de desarrollo y producción.
- **Diseño Limpio y Mantenible:** La aplicación rigurosa de patrones de diseño (Composite, Facade, Adapter, Strategy, Observer, Template Method, Factory) erradicó antipatrones comunes, facilitando la mantenibilidad a largo plazo de las más de 32 pantallas y 53 controladores desarrollados.

10.2 Recomendaciones Técnicas para el Equipo

1	**Fidelidad del Diseño Gráfico** Mantener el catálogo de Figma como la única fuente de verdad gráfica previa a cualquier adición funcional en los repositorios.
2	**Aislamiento de la Capa de Servicios** Toda interacción con nuevos endpoints de la API debe canalizarse exclusivamente a través de la carpeta `services/`, preservando la fachada que centraliza la seguridad.
3	**Validación Continua en Móvil** Continuar ejecutando compilaciones periódicas del APK de Android con Expo para detectar tempranamente cualquier inconsistencia entre librerías nativas.
4	**Cobertura y Calidad de Código** Mantener la política de commits atómicos y pruebas unitarias con JaCoCo en el backend con una cobertura mínima del 75% en servicios críticos.

11. ANEXOS Y MATRIZ DE TRAZABILIDAD (SRS vs. CÓDIGO)

A continuación se certifica la trazabilidad directa entre los Requerimientos Funcionales del SRS, los módulos de código del frontend y los controladores del backend:

Requerimiento SRS	Módulo Web	Módulo Móvil	Controlador Backend
RF01-RF08: Autenticación, 2FA y Roles	src/modules/auth/ (LoginPage, Verify2FA)	modules/auth/ (useAuth, useSocialAuth)	AuthController.java, SocialAuthController.java
RF09-RF15: Catálogo de Flota y Búsqueda	src/modules/catalog/ (CatalogPage, VehicleDetails)	modules/catalog/ (useCatalog)	VehicleController.java, CatalogController.java
RF16-RF24: Wizard de Reservas y Sedes	src/modules/reservations/ (ReservationFlowPage)	modules/reservation/ (useReservationWizard)	ReservationController.java, BranchController.java
RF25-RF30: Pasarela Wompi y Efectivo	src/modules/payments/ (WompiPaymentStrategy)	modules/reservation/ (CheckoutScreen)	PaymentWebhookController.java, CashPaymentController.java
RF31-RF36: Contratos PDF y PIN de Entrega	src/modules/contracts/ (ContractSigningPage)	services/pdfService.ts (Expo Print)	ContractController.java, DeliveryDriverActionController.java
RF37-RF52: Panel Admin, Flota y Reportes	src/modules/admin/ (VehicleManagement, Reports)	Bloqueado por diseño en app móvil	AdminVehicleController.java, AdminReportController.java

Fin de la documentación técnica oficial de Arquitectura de Software para Drivique Location.